

TIC TAC TOE GAME

21CSC101T/OBJECT ORIENTED DESIGN AND PROGRAMMING PROJECT REPORT

Submitted by

ASHWIN KARTHI H	RA2411003020192
C VARUN	RA2411003020199
G M BHOOPESH	RA2411003020252

Under the guidance of

Dr. K. SUJATHA SARAVANAN

(Assistant Professor, Department of Computer Science and Engineering)

II SEMESTER / I YEAR

COMPUTER SCIENCE AND ENGINEERING

FACULTY OF ENGINEERING AND TECHNOLOGY



**SRM INSTITUTE OF SCIENCE AND TECHNOLOGY
RAMAPURAM, CHENNAI**

APRIL 2025

SRM INSTITUTE OF SCIENCE AND TECHNOLOGY
(Deemed to be University U/S 3 of UGC Act, 1956)

BONAFIDE CERTIFICATE

Certified that this project report titled **TIC TAC TOE GAME**
is the bonafide work of **ASHWINKARTHI H (RA2411003020192)**,
C VARUN(RA2411003020199), and **G M BHOOPESH(RA2411003020252)**, who
carried out the project work under my supervision. This project work confirms
to 21CSC101T /Object Oriented Design and Programming, II Semester, I year, 2025.

SIGNATURE

DR.K.SUJATHA SARAVANAN
M.E.,Ph.D,Assistant Professor

Computer Science and Engineering,

SRM Institute of Science and Technology,
Ramapuram, Chennai.

**SRM INSTITUTE OF SCIENCE AND TECHNOLOGY
RAMAPURAM, CHENNAI**

DECLARATION

We hereby declare that the entire work contained in this project report titled

“TIC TAC TOE ” has been carried out by **ASHWIN KARTHI H**
(RA2411003020192), C VARUN (RA2411003020199), and

G M BHOOPESH (RA2411003020252) at SRM Institute of Science and Technology,
Ramapuram, Chennai, under the guidance of **DR.K.SUJATHA SARAVANAN**,
Assistant professor, Department of Computer Science and Engineering.

Place: Chennai

Date:

ASHWIN KARTHI H

C VARUN

G M BHOOPESH

INDEX

、	CONTENT	PAGE NUMBER
1	ABSTRACT	5-6
2	INTRODUCTION	7
3	PROBLEM STATEMENT	8-9
4	OBJECTIVE	10
5	SCOPE OF THE PROJECT	11
6	EXISTING SYSTEM	12
7	PROPOSED SYSTEM	13
8	MODULES DESCRIPTION	14
9	UML DIAGRAMS	15-19
10	IMPLEMENTATION & CODING	20-23
11	SCREENSHOTS	24-25
12	CONCLUSION	26
13	APPENDIX	27-28

1.ABSTRACT

1.1.Objective:

To design and implement the Tic-Tac-Toe game using object-oriented principles and UML diagrams.

1.2.Problem Statement:

Representing game logic, state management, and player interactions in a structured and scalable way

1.3.UML Diagrams Used:

1. **Use Case Diagram** – Shows interaction between players and the game system.
2. **Class Diagram** – Models core components like Game, Board, Player.
3. **Sequence Diagram** – Describes the flow of moves and game state updates.

1.4.Key Components:

1. **Game:** Controls game flow and turn handling.
2. **Board:** Maintains game state and checks for win/draw.
3. **Player:** Represents a human (or AI) player and their symbol (X or O).

1.5.Solutions Provided:

1. Modular class-based design for clean separation of logic.
2. Reusable components for potential AI
3. Input validation and clear win/draw conditions implemented.

1.6.Outcome:

1. A maintainable and extendable C++ application.
2. Real-world application of UML in game development.

2.INTRODUCTION

Tic Tac Toe, also known as "Noughts and Crosses," is one of the most elementary games played by children and adults alike. It is a strategic game that requires foresight and logical thinking. The objective is to complete a row, column, or diagonal of matching symbols before your opponent.

The simplicity of the game makes it an ideal candidate for digital simulation. By programming Tic Tac Toe in C++, we engage directly with fundamental programming concepts including arrays, functions, loops, conditional statements, and input/output handling. This project leverages these concepts to simulate a game that is both familiar and educational.

In recent years, Tic Tac Toe has become a popular project for students and new programmers as a way to apply basic programming concepts. It offers a hands-on way to learn how to design logic systems, manage game states, and handle user inputs. Because the rules are easy to understand, the focus remains on writing clean, functional code that accurately represents the game logic.

Furthermore, the development of a Tic Tac Toe game provides a foundation for understanding more advanced topics such as AI. Implementing a computer opponent introduces concepts like decision-making algorithms, including the minimax strategy. Whether played for fun or developed as a software project, Tic Tac Toe continues to be a valuable learning tool and a nostalgic reminder of simple, strategic gameplay.

This report explains the structure and functioning of the Tic Tac Toe project. It discusses the problem it solves, the objectives it meets, the approach used to design and implement it, and the technical details of each module involved. The final output is a fully functional console-based game that allows two players to play multiple rounds until one emerges as the winner or a draw is declared..

2.PROBLEM STATEMENT

Despite its simplicity, developing a digital version of Tic Tac Toe presents a unique opportunity to apply and integrate various programming concepts in a real-world context. Many learners understand theory but face difficulty in actual implementation. This project attempts to fill that gap.

The primary problem addressed by the Tic Tac Toe game is the creation of a functional and interactive two-player game that follows a defined set of rules. The challenge lies in designing a system where two users can alternate turns seamlessly while ensuring the game logic correctly identifies valid moves, detects win conditions, and handles draw situations. The game must also prevent illegal actions, such as selecting an already occupied cell, and offer a clear visual representation of the current game state.

Another key aspect of the problem is implementing the logic to evaluate the outcome after each move. The system must be able to scan the 3x3 grid for all possible winning combinations—rows, columns, and diagonals—after each turn. Additionally, it should determine when the board is full without any winners, resulting in a draw. Ensuring accurate and efficient checking of these conditions is essential for the game's correctness and smooth gameplay.

If the game includes a single-player mode against the computer, the problem becomes more complex with the addition of artificial intelligence. The challenge now involves creating a nontrivial opponent that can make strategic decisions. This may include implementing algorithms that allow the computer to play either randomly or optimally using strategies such as minimax. The aim is to provide a challenging experience for the player while maintaining the core rules and fairness of the game.

There is a clear need for beginner-level projects that are simple to understand, yet comprehensive enough to cover core programming principles. Traditional teaching methods may not provide adequate experience in handling dynamic input/output or error handling during program execution.

By building the Tic Tac Toe game from scratch, students gain the opportunity to:

- Handle user input with validation.
- Build game logic with conditionals.
- Manage control flow and modularity.
- Debug logical errors and handle exceptions.

The problem this project solves is the lack of small-scale, concept-focused projects that give students the ability to apply what they learn in classrooms to create something tangible and enjoyable.

3.OBJECTIVE

The primary objectives of the Tic Tac Toe Game project are:

1. **Create a 3x3 game board** that clearly displays the current state of the game.
2. **Allow two players to take alternate turns**, marking "X" and "O" on the grid.
3. **Ensure valid move input**, preventing players from selecting already occupied cells.
4. **Check for a win condition** after every move—horizontal, vertical, or diagonal alignment
5. **Detect a draw condition** when all cells are filled without a winner.
6. **Provide a user-friendly interface** for ease of play, either via console, GUI, or web.
7. **Display messages** to indicate the current player's turn, the winner, or a draw.
8. **Reset the game** after it ends so players can play again without restarting the application.
9. **Maintain basic error handling** for invalid inputs or unexpected actions.
10. **Track and display game status** in real-time throughout the session.
11. **Keep the game logic modular**, separating game engine from user interface for better readability and maintenance.
12. **Allow player vs. computer mode**, where one side is controlled by a simple AI (optional).
13. **Ensure cross-platform compatibility**, so the game works on different devices or browsers (if web-based).
14. **Create a visually appealing design** (if graphical), including intuitive layout, animations, or sound effects (optional).

4.SCOPE OF THE PROJECT

Game Functionality :

1. Implements a 2-player Tic-Tac-Toe game.
2. Allows players to input moves on a 3x3 grid.
3. Checks for win or draw conditions after each move.
4. Displays the board state after every turn.

Design and Modeling:

1. Uses UML diagrams to plan and model the system
2. Use Case Diagram for user interactions.
3. Class Diagram for object structure and relationships.
4. Sequence Diagram for game flow.
5. Emphasizes object-oriented principles like encapsulation and modularity.

Development Platform:

1. Implemented in C++ for console-based interaction.
2. Follows clean coding practices for readability and future extension

Educational Purpose:

1. Helps learners understand software design using UML.
2. Bridges the gap between design (UML) and implementation (C++).

Future Enhancements (Out of Scope for now but possible):

1. Single-player mode with AI using minimax algorithm.
2. GUI-based version using graphics libraries.
3. Online multiplayer functionality.

5.EXISTING SYSTEM

Numerous Tic Tac Toe implementations already exist in web apps, mobile games, and learning platforms. Some use artificial intelligence, while others focus on single or multiplayer functionality. However, many of these systems do not provide access to their internal logic, making them unsuitable for educational dissection. In **console-based versions**, the game typically operates through text-based interaction. Players are prompted to enter the row and column number where they wish to place their mark. The console then refreshes to display the updated grid. While minimalistic, these versions serve as excellent introductory projects for learning control flow, loops, and arrays in programming.

Some existing systems offer **Graphical User Interfaces (GUIs)** using libraries like Java Swing, Tkinter (Python), or frameworks like React and Flutter. These versions enhance the user experience by allowing players to click on cells instead of typing coordinates. The board is visually more appealing, and messages are displayed clearly to indicate player turns, invalid moves, or game results.

A number of existing systems have introduced **basic AI for single-player mode**, allowing users to play against the computer. In simpler systems, the computer may make random moves. More advanced systems implement decision-making algorithms like **Minimax**, which ensures the AI either wins or forces a draw. These versions add a layer of complexity to the game and make it more engaging for solo players.

The existing Tic Tac Toe systems are generally **limited in scope** to basic functionality, offering only one game mode and lacking additional features like score tracking, difficulty levels, player profiles, or game statistics. However, this simplicity makes them lightweight and easy to develop, maintain, and extend.

On web-based platforms, Tic Tac Toe games are often made with **HTML, CSS, and JavaScript**, making them accessible via browsers without the need for installation. Some versions even support multiplayer over the internet using socket-based communication. These implementations can be found on various coding tutorial websites and open-source platforms like GitHub.

6.PROPOSED SYSTEM

The proposed system is a console-based 2-player Tic-Tac-Toe game designed using object-oriented principles and UML diagrams.

1. It introduces a modular class-based structure to separate game logic, board handling, and player interaction.
2. The system utilizes UML diagrams for pre-implementation design:
3. Use Case Diagram to outline player actions.
4. Class Diagram to define object structure and relationships.
5. Sequence Diagram to map the order of gameplay interactions.
6. The game is developed in C++, ensuring a balance between performance and structure.

Features include:

1. Real-time board updates.
2. Player input handling and move validation.
3. Automated win and draw condition checks.
4. Turn-based switching between Player X and Player O.
5. The system is designed to be scalable, allowing future extensions like AI integration or GUI enhancement.

7.MODULES DESCRIPTION

1. **Display Module:** Displays the 3x3 board after every turn, ensuring users know the current state of the game.
2. **Input Module:** Takes user input (slot number), validates it, and passes it to the game logic.
3. **Game Logic Module:** Handles placement of markers, win/draw condition checking, and manages rounds.
4. **Swap Module:** Switches player turn and marker between 'X' and 'O'.
5. **Control Flow Module:** Manages the overall structure of the game, ensuring proper execution of the game loop
6. **AI Module:** Handles computer moves if playing in single-player mode. Generates random or strategic moves using algorithms like Minimax.
7. **Scoreboard Module:** Racks the number of games won by each player. Displays scores after each round. Allows users to reset or continue playing.

8.UML DIAGRAMS

8.1.USE-CASE DIAGRAM:

Use Case Diagram

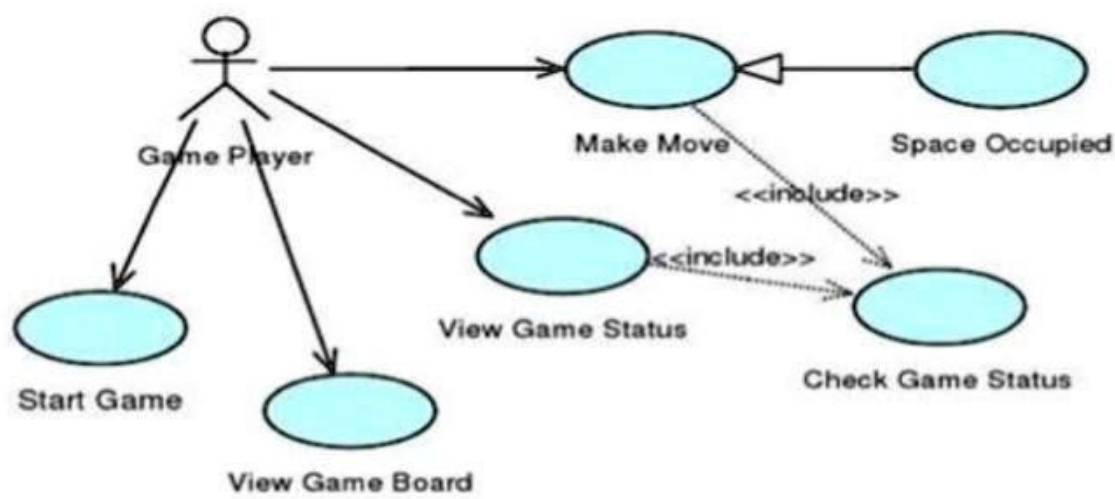


Fig 8.1

8.2.CLASS DIAGRAM

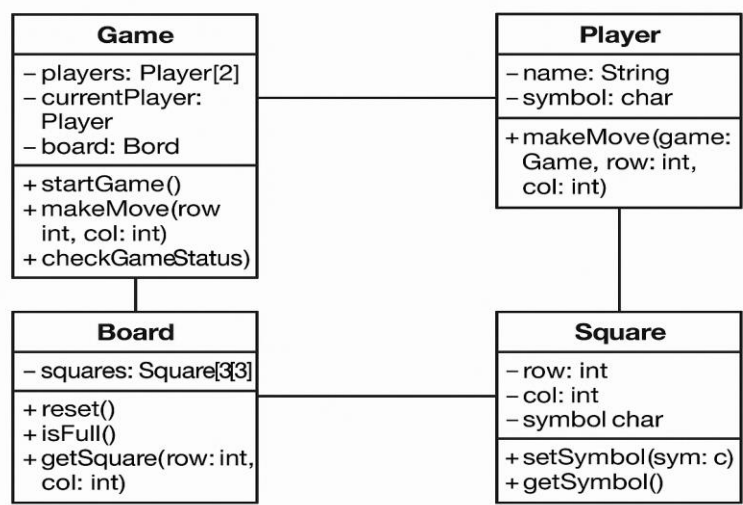


Fig 8.2

8.3. SEQUENCE DIAGRAM

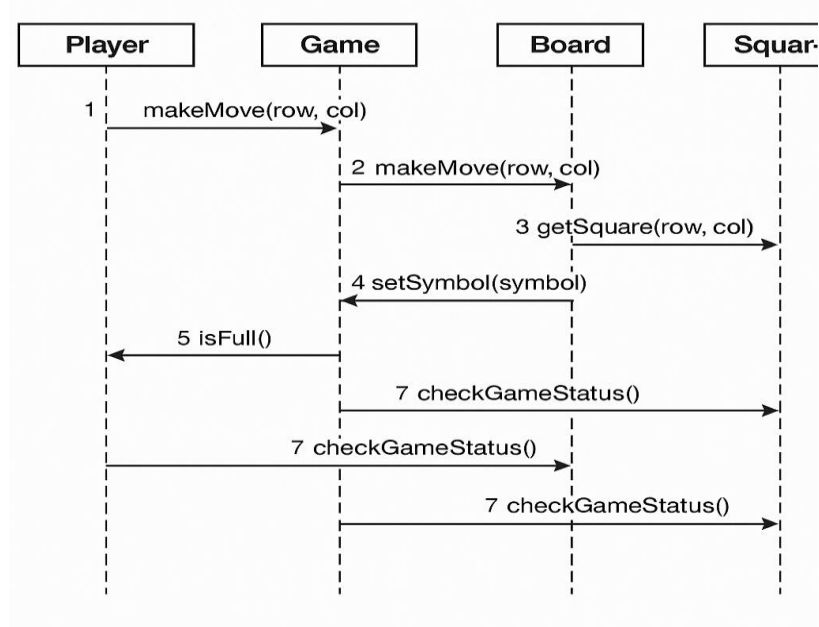


Fig 8.3

8.4. ACTIVITY DIAGRAM

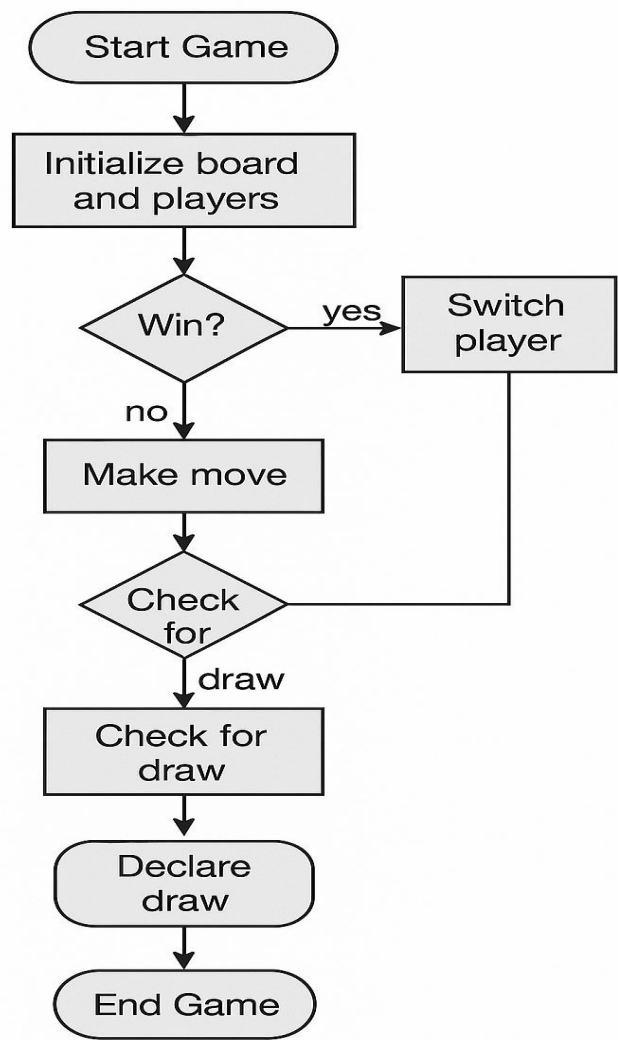


Fig 8.4

8.5. STATE CHART DIAGRAM :

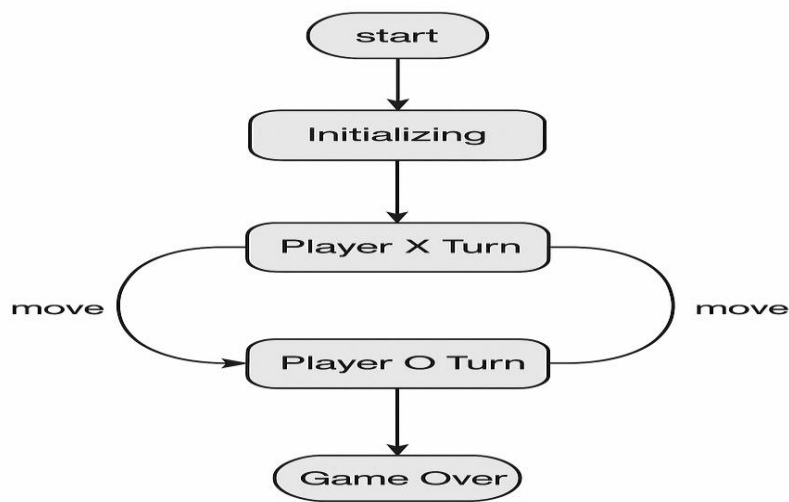


Fig 8.5

6.COMPONENT DIAGRAM

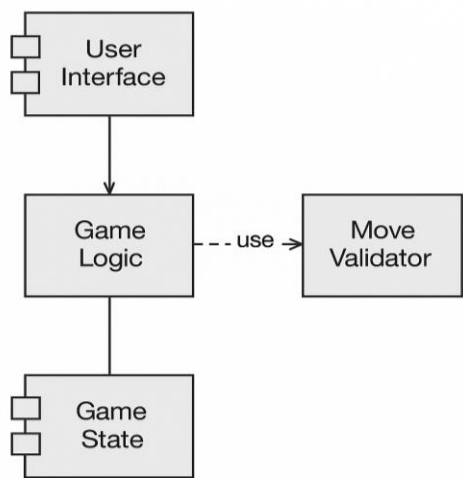


Fig 8.6

7. DEPOLYMENT DIAGRAM

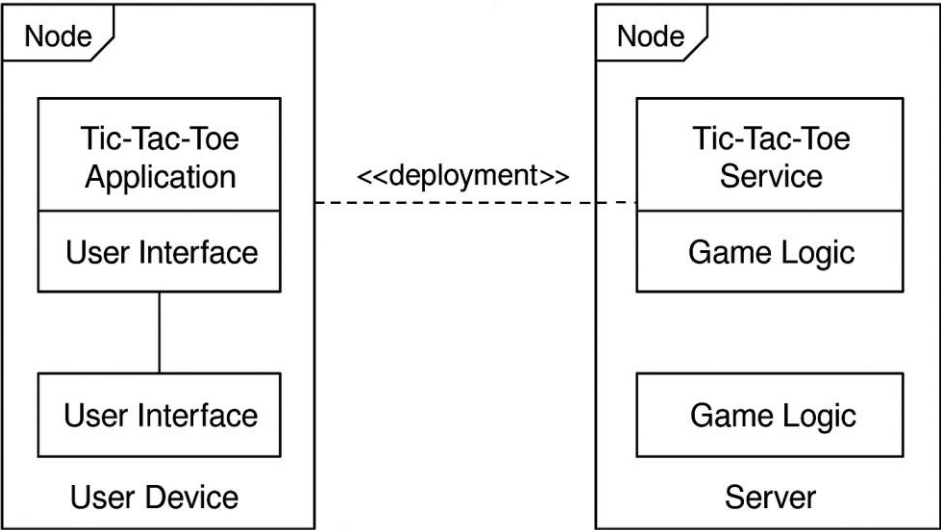


Fig 8.7

9.IMPLEMENTATION AND CODING

```
#include <iostream>
using namespace std;
char board[3][3];
void initBoard() {
    for (int i = 0; i < 3; ++i)
        for (int j = 0; j < 3; ++j)
            board[i][j] = ' ';
}

void printBoard() {
    cout << "\n";
    for (int i = 0; i < 3; ++i) {
        cout << " " <<
            board[i][0] << " | " <<
            board[i][1] << " | " <<
            board[i][2] << "\n";
        if (i < 2)
            cout << "---|---|---\n";
    }
    cout << "\n";
}

bool isWinner(char
player) {
    // Rows and columns
    for (int i = 0; i < 3; ++i) {
        if ((board[i][0] == player
&& board[i][1] == player
&& board[i][2] ==
player) ||
```

```

(board[0][i] == player
&& board[1][i] == player
&& board[2][i] ==
player))
return true;
}

// Diagonals
if ((board[0][0] == player && board[1][1] ==player && board[2][2] == player) ||
(board[0][2] == player
&& board[1][1] ==
player && board[2][0]
== player))
return true;

return false;
}

bool isDraw() {
for (int i = 0; i < 3; ++i)
for (int j = 0; j < 3; ++j)
if (board[i][j] == ' ')
return false;
return true;
}

int main() {
int row, col;
char currentPlayer = 'X';
initBoard();

while (true) {
printBoard();

```

```
cout << "Player " <<
currentPlayer << ", enter
row and column (0-2): ";
cin >> row >> col;
```

```
if (row < 0 || row > 2 ||
col < 0 || col > 2 ||
board[row][col] != ' ') {
cout << "Invalid move.
Try again.\n";
continue;
}
```

```
board[row][col] =
currentPlayer;
```

```
if
(isWinner(currentPlayer)
) {
printBoard();
cout << "Player " <<
currentPlayer << "
wins!\n";
break;
} else if (isDraw()) {
printBoard();
cout << "It's a draw!\n";
break;
}
```

```
currentPlayer =
(currentPlayer == 'X') ?
'O' : 'X';
```

```
}
```

```
return 0;
```

```
}
```

10.SCREENSHOTS

Player 1, choose your marker (X or O): X

```
1 | 2 | 3
---|---|---
4 | 5 | 6
---|---|---
7 | 8 | 9
```

Player 1, enter your slot (1-9): 1

```
X | 2 | 3
---|---|---
4 | 5 | 6
---|---|---
7 | 8 | 9
```

Player 2, enter your slot (1-9): 5

```
X | 2 | 3
---|---|---
4 | 0 | 6
---|---|---
7 | 8 | 9
```

Fig 10.1

Player 1, enter your slot (1-9): 2

X		X		3
---		---		---
4		0		6
---		---		---
7		8		9

Player 2, enter your slot (1-9): 9

X		X		3
---		---		---
4		0		6
---		---		---
7		8		0

Player 1, enter your slot (1-9): 3

X		X		X
---		---		---
4		0		6
---		---		---
7		8		0

Player 1 wins!

Fig 10.2

11.CONCLUSION

In conclusion, the Tic Tac Toe game serves as a foundational project that combines both logical thinking and programming skills. Its simplicity makes it ideal for beginners to learn core programming concepts such as loops, conditionals, arrays, and user interaction. By building this game, developers gain practical experience in creating interactive applications with real-time feedback and decision-making logic.

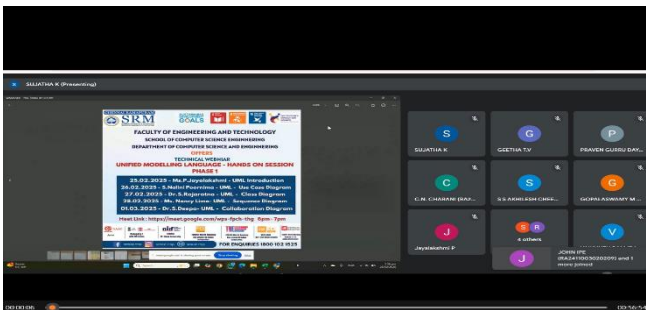
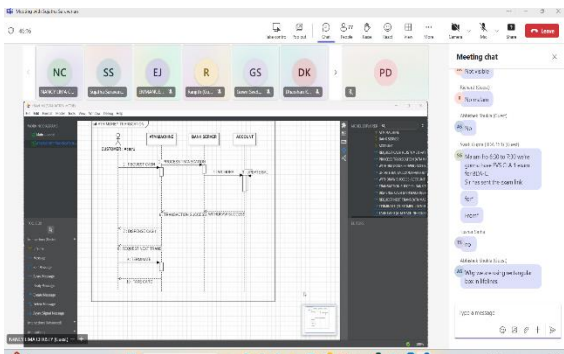
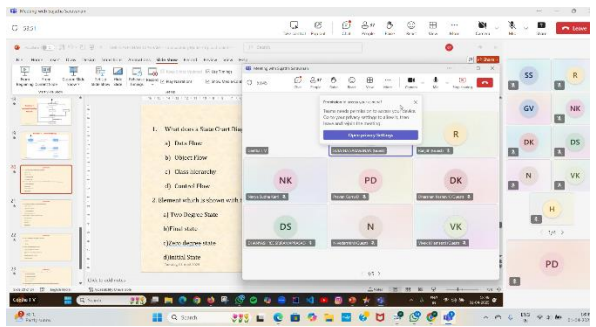
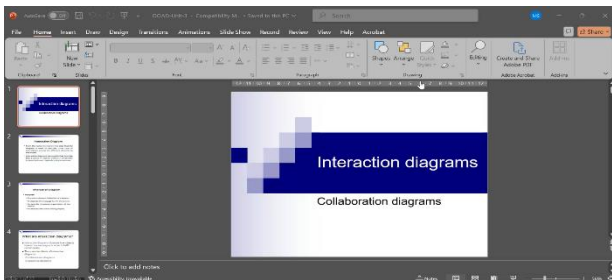
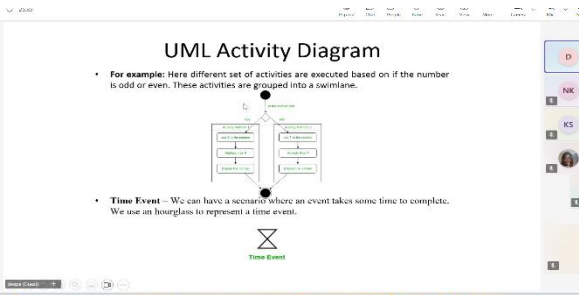
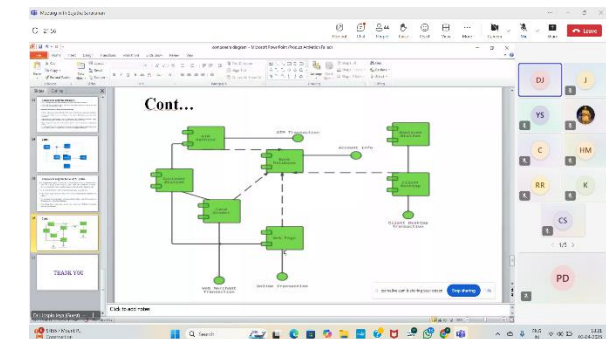
The game's implementation also opens up opportunities to explore more advanced topics like artificial intelligence, game algorithms (such as Minimax), and GUI development. With options to expand the game into single-player or online multiplayer modes, Tic Tac Toe offers a scalable platform for further development and innovation. It not only helps

The Tic Tac Toe Game project achieves its objective of demonstrating key programming principles in a practical, engaging manner. By recreating a simple game, we explore how to design, implement, and manage an interactive software system. The project reflects real-world programming flow, encouraging best practices such as modularity, input validation, and clean output formatting.

While basic in scope, the project leaves room for enhancements such as an AI opponent, graphical UI, and advanced scoring systems. It provides a foundational learning experience for any computer science student beginning their journey in software development.

Overall, Tic Tac Toe remains a timeless and engaging project that balances simplicity with educational value. Whether developed as a console app, web game, or mobile application, it encourages creativity and critical thinking. As a result, it continues to be a popular choice for students, hobbyists, and educators in the field of software development and computer science.

12.APPENDIXES





CERTIFICATE OF ACHIEVEMENT

Congratulations,

C VARUN

on successfully completing the course,
BUILD YOUR OWN STATIC WEBSITE



HTML



CSS



Bootstrap

Issue Date:
February 11, 2025

Member of
NASSCOM®



Program designed by top alumni from
Stanford University Google IIT Bombay amazon IIT Delhi Microsoft

Verify at: <https://certificates.ccbp.in/academy/static-website?id=WZUAAOPFBS>

Rahul A
CEO, NxtWave